Week 23 (Days 155-161)

Status: Days 155-161 are written in full. This is Month 6's deployment week: ONNX export and ONNX Runtime tuning, int8 quantization, pruning and distillation, memory and KV-cache budgeting, shipping a model to Android, a small inference engine written in Rust, and a weekly project that holds every variant to one latency/memory/accuracy/size budget. Every number comes from `code/week-23/day155_*.py` to `day161_*.py`, each run on this box, with logs in `code/week-23/out/`. Shared helpers (the digits MLP, ONNX export, a median-latency bench, RSS) are in `code/week-23/_dep.py`. The Rust crate is `code/week-23/tiny_infer/`, and the Android snippets are in `code/week-23/android_untested/`.

Retitled from the original SYLLABUS rows. The rows for 155-158 were GPU topics (CUDA, Triton, FlashAttention, distributed training). There is no GPU here, and Week 22 already covered those ideas as UNTESTED sketches, so this week follows the deployment brief instead. The SYLLABUS rows are updated to match. The old Day 159/160 topics (ONNX Runtime, on-device Android) are now Days 155 and 159.

The model. The model is a 64-512-512-10 ReLU MLP trained on scikit-learn's bundled digits dataset (1,797 8x8 images, all offline). It has 301,066 parameters and reaches 0.9741 test accuracy on a 540-row stratified test split. This is a deliberately tiny model. It makes every experiment run in seconds, and it keeps the deployment mechanics visible. It also has a cost, stated wherever it matters: digits is easy, so int8 never changes accuracy here, and distillation does not help. Day 158 switches to the real SmolLM2-135M-Instruct config for the LLM memory arithmetic.

Not available, and labelled wherever it appears: - There is **no Android SDK/NDK, Gradle, emulator or phone**. Everything in `android_untested/` is **UNTESTED**: written from the official pages, never compiled. The model *files* were produced and run here, on x86-64 Linux, with each runtime's Python API: ORT-format `.ort` files and ExecuTorch `.pte` files (portable and XNNPACK). That is real evidence the files are valid. It is not evidence about speed on a phone. - **LiteRT:** the `ai-edge-litert` 2.3.0 interpreter is installed and imports. Converting PyTorch to `.tflite` needs a separate converter package that is not installed here, so LiteRT conversion and runs are **UNTESTED**. - **llama.cpp:** there is no C++ compiler on this box and no GGUF model was downloaded, so it is **UNTESTED** (notes only). - **No GPU:** see the compute note.

Compute note. The CPU is an 8-core Xeon with AVX-512, AVX-512 VNNI, AVX-VNNI and AMX-INT8 (flags printed in the Day 156 log), with 15 GB RAM. Every latency below is from this server CPU at microsecond scale. A phone's big cores have less cache and lower clocks, and they throttle under heat, so absolute numbers will be different there. Re-measure on the device; the *method* is what transfers. A GPU would change Days 155-156 a little: ORT would offer the CUDA/TensorRT execution providers and fp16/int8 tensor cores, but at batch 1 on a model this small, launch overhead dominates and a GPU would probably not help. Day 158 would change a lot, since prefill on a 135M LLM is compute-bound, which suits a GPU. On phones, the equivalent accelerators are the GPU and NPU delegates/backends that LiteRT and ExecuTorch list (Vulkan, Qualcomm, MediaTek), which are **UNTESTED** here.

Sources (all opened and checked this week; links in `docs/resources.md`): the ONNX Runtime docs on graph optimisations, threading, profiling, quantization, ORT-format models, install (Android) and the Java API; the ExecuTorch "Using ExecuTorch on Android" and getting-started pages; Google's LiteRT Android page; llama.cpp's `docs/android.md`; and the PyTorch pruning tutorial. Explanations here are original, not copied.

Day 155: ONNX export and ONNX Runtime tuning: graph optimisations, threads and the profiler

Review today: Day 154 (+1), Day 152 (+3), Day 148 (+7), Day 134 (+21; quick 15-min skim of its self-check).

Block 1: Why export at all (25 min)

A trained PyTorch model is Python objects plus a training framework of several hundred megabytes. A deployed model should be a *file*, plus a *runtime* that is small, fast and stable across versions. ONNX is a file format for that: a graph of standard operators (Gemm, Relu, Conv, MatMul...) with the weights inside, tagged with an *opset* version that pins what each operator means. ONNX Runtime (ORT) is one engine that executes such graphs on many targets: server CPUs, GPUs, browsers, and phones through ONNX Runtime Mobile. Week 21 (Day 144) already used ORT as a serving engine and found it about 4.2x faster per call than eager PyTorch, but only about 1.15x faster once wrapped in HTTP. That lesson still holds: tune the engine only after you know

the engine is your bottleneck. This week the engine *is* the product, because on a phone there is no HTTP hop to hide behind.

Export works by running the model once on an example input and recording the operators it executes (the classic tracer; `dynamo=False` in `_dep.export_onnx`). Two decisions matter. **Dynamic axes:** without them, the file is fixed to the example's batch size. We mark axis 0 as `batch`, so the same file serves batch 1 and batch 256. **Opset:** we pin 17. A newer opset can express more, but older runtimes (old app builds) may not load it.

Block 2: Run it (35 min)

`python day155_onnx_ort.py` exports the MLP, checks it with `onnx.checker`, compares it with PyTorch, then times four things.

Verified (out/day155_onnx_ort.log): - The export is a 1,205 kB file with opset 17 and exactly five nodes: `Gemm, Relu, Gemm, Relu, Gemm`. On all 540 test rows, the maximum absolute logit difference against PyTorch is 0.00e+00, argmax agreement is 1.0000, and ORT accuracy is 0.9741. The numbers are identical, not just close: the same fp32 GEMM math in the same order. - **Graph optimisation levels (1 thread, median µs per call):** DISABLE_ALL 15.1 at batch 1 and 686.3 at batch 256; BASIC 19.7 / 690.6; EXTENDED 15.6 / 763.4; ALL 14.4 / 680.5. Saving each optimised graph shows what changed. From EXTENDED upwards, ORT fused each Gemm+Relu pair into a contrib `FusedGemm` op, leaving `['FusedGemm', 'FusedGemm', 'Gemm']`. - **Thread sweep (ENABLE_ALL):** with 1/2/4/8 intra-op threads, batch 1 took 15.6 / 12.1 / 11.3 / 11.7 µs and batch 256 took 676.3 / 463.0 / 325.1 / 313.9 µs. Eager PyTorch with 8 threads took 23.2 µs at batch 1 and 255.7 µs at batch 256. - `ORT_PARALLEL` execution mode at batch 1 took 18.7 µs. A chain graph has no independent branches, so inter-op parallelism has nothing to do. - The ORT profiler (JSON trace in `out/`, 50 runs at batch 256) attributes 97% of kernel time to `FusedGemm` and 3% to the final `Gemm`.

Block 3: Read the numbers honestly (30 min)

Fusion did not buy measurable speed here. The level differences are a few µs at batch 1, and they move between runs. The EXTENDED batch-256 figure of 763.4 is slower than ALL's 680.5, although both levels produced the same saved graph, so that gap is noise and not a real effect. Why so little gain? Fusing a Relu into a Gemm saves one pass over a 256x512 activation tensor, and that pass is tiny next to the GEMM itself (the profiler's 97%). Fusions pay off when the fused-away ops are a large share of the time: layer norms, GELUs, attention patterns in transformers, Conv+BatchNorm in CNNs. The rule is simple: leave ENABLE_ALL on (it is the default), and don't expect miracles on a GEMM-bound MLP.

Threads help batch 256, not batch 1. Going from 1 to 4 threads cut batch-256 latency from 676.3 to 325.1 µs, about 2.1x. Going from 4 to 8 added almost nothing (313.9). At batch 1 the best was 11.3 µs at 4 threads against 15.6 at 1: a small gain that costs four cores. On a phone, that trade is usually wrong. More cores at full clock drain the battery and heat the device, and the app's UI thread competes for the same cores. A common on-device default is one or two intra-op threads for small models. Measure the gain before spending more.

Eager PyTorch wins at batch 256 with 8 threads (255.7 µs). That is a fair observation, not a failure of ORT. PyTorch's CPU GEMM path is very good at large batches on AVX-512. ORT's case for deployment is its small runtime, stable file format, mobile builds and execution providers, *and* lower per-call overhead at batch 1 (14.4 vs 23.2 µs here).

Warning in the raw output. When you save a graph optimised at ALL, ORT warns that it may contain hardware-specific optimisations (the NCHWc layout transformer) and should only be used on the same kind of machine. Lesson: ship the *unoptimised* or BASIC/EXTENDED-optimised model to a device, and let the device's runtime apply its own hardware-specific passes.

Block 4: Self-check (20 min)

1. Why did ENABLE_EXTENDED and ENABLE_ALL produce the same graph for this model? *Both apply the Gemm+Relu fusion. The extra passes in ALL are layout transforms that only matter for some op types (e.g. convolutions), and this MLP has none.*
2. Batch 1 at 1 thread took 15.6 µs and at 4 threads 11.3 µs. Would you ship 4 threads on a phone? *Probably not. A 4.3 µs gain is invisible to a user, and it costs three more cores' power and contention. Re-measure on the device first.*
3. What does the profiler's 97% for FusedGemm tell you about further graph tuning? *That the remaining 3% is the ceiling for non-GEMM tuning. Speed must come from the GEMMs themselves: fewer FLOPs (pruning) or cheaper FLOPs (int8).*
4. Why did the export diff come out exactly 0? *Same operators, same fp32 arithmetic, same accumulation order on this CPU. Different kernels or a different machine could give differences around 1e-6. Always check with a tolerance, not equality.*

Block 5: Practice (30 min)

Re-export with `dynamic_axes` removed and try `s.run` at batch 256: read the error message. Then set `so.enable_profiling` in your own session, open the JSON in a trace viewer (Chrome's `about://tracing` or Perfetto), and find the `FusedGemm` spans.

Extra notes

- `intra_op_num_threads` splits one operator across threads. `inter_op_num_threads` (in PARALLEL mode only) runs independent operators at the same time. Most models want intra; graphs with real branches (multi-head models, ensembles) can use inter.
- ORT's thread pool spins threads for a short while after work, to cut wake-up latency. That costs CPU while idle. The ORT threading docs show how to turn spinning off, which matters for a phone's battery.
- `onnx.checker.check_model` checks structure, not numerics. The comparison against PyTorch on real data is the test that counts.

Going deeper

Read ORT's graph optimisation page and list the fusions in each level. Then export a small transformer block (Week 13) and look for `LayerNormalization`, `Gelu` and attention fusions in the saved graph. Those are where fusion gains come from.

Interview-style question (15 min)

"Our ONNX model is 2x slower in production than on my laptop." Walk through it: same ORT version and execution provider? Same threads, and is the container's CPU quota lower than the visible core count? Is graph optimisation actually on? Is the batch size the same? Is the first call (warm-up) being timed? Is a hardware-specific optimised graph being shipped to a different CPU?

Common mistakes

- Timing the first `run`, which includes lazy allocation.
- Shipping a graph saved at ENABLE_ALL to a different machine (the warning above).
- Forgetting dynamic axes, then padding everything to the export batch size.
- Judging threads at batch 256 when the product runs batch 1.

What to do next

Day 156 makes the GEMMs cheaper instead of fewer: int8 quantization, measured on accuracy, size and latency at the same time.

Day 156: Quantization for deployment: int8 dynamic and static, measured honestly

Review today: Day 155 (+1), Day 153 (+3), Day 149 (+7), Day 135 (+21; quick 15-min skim of its self-check).

Block 1: What int8 quantization is (30 min)

Quantization stores a tensor as small integers plus a scale (and optionally a zero point): real ~= scale x (q - zero_point). With int8 weights, the model file is about 4x smaller than fp32, memory traffic drops by the same factor, and the CPU can use integer dot-product instructions. On x86 those are VNNI (and AMX); on Arm phones, the dot-product extensions. All are present on this CPU: the log lists `avx512bw, avx_vnni, avx512_vnni, amx_int8`. Integer GEMMs accumulate in int32 and convert back to float at the end of each layer.

The flavours differ in *when* activation scales are decided: - **Dynamic quantization:** weights are quantized offline. Each activation tensor's range is measured at run time, on every call, and quantized on the fly. There is no calibration data and little risk. The per-call range computation costs a little. - **Static quantization:** activation ranges are fixed ahead of time by running a *calibration* set through the model. It is faster at run time (no range scan), but it is only as good as the calibration data. A camera app calibrated on daylight photos may clip at night. - **Format:** ORT's static path can write QDQ (QuantizeLinear/DequantizeLinear pairs around float ops, which the runtime then fuses) or QOperator (dedicated integer ops). QDQ is the format most converters and mobile backends expect. - **Granularity:** per-tensor uses one scale for a whole weight matrix; per-channel uses one scale per output channel. Per-channel is usually more accurate when rows have very different magnitudes.

Quantization-aware training (QAT) simulates rounding during training, so the model learns to tolerate it. It is SYLLABUS Day 162's subject; today is post-training quantization only.

Block 2: Run it (35 min)

`python day156_quantize.py` makes five ONNX variants plus a PyTorch one. The calibration set is 200 rows of *training* data, never test data.

Verified (out/day156_quantize.log):

variant	size kB	acc	agree	max abs ?logit	b1 µs (1 thread)	b256 µs (1 thread)	b256 µs (4 threads)
fp32	1205	0.9741	1.000	0.000	15.7	680.3	325.8
ORT dynamic int8	307	0.9741	1.000	0.132	11.0	211.8	185.0
ORT static QDQ per-tensor MinMax	308	0.9741	1.000	0.935	10.6	155.6	169.9
ORT static QDQ per-channel MinMax	321	0.9741	1.000	0.935	16.7	183.4	179.6
ORT static QDQ per-channel Percentile	321	0.9741	1.000	0.941	16.6	184.0	173.1

- PyTorch eager fp32 (4 threads): 22.4 µs at batch 1, 305.1 µs at batch 256, with a 1,207 kB state_dict.
- PyTorch dynamic qint8 (4 threads, `x86` engine): accuracy 0.9741, agreement 1.000, max abs ?logit 0.183, 52.2 µs at batch 1, 267.5 µs at batch 256, with a 309 kB state_dict.
- ORT printed a warning recommending its pre-processing step (shape inference and optimisation before quantizing). We skipped it. On this five-node graph it has little to do; on real models, run it.

Block 3: What the table really says (30 min)

Size: a clean 4x. 1,205 kB went down to 307-321 kB. Per-channel files are slightly bigger because they store more scales.

Accuracy did not move, and that is a limitation of the test, not proof that int8 is free. Every variant scored 0.9741, with 100% argmax agreement with fp32. The *logits* did move: static quantization changed them by up to 0.94, dynamic by 0.13. Digits' decision margins are wide, so these perturbations never flip a class. On a harder task (a 1,000-class model, or an LLM's next-token distribution where the top two tokens are close), a 0.94 logit error *will* flip decisions. Report the logit error as well as accuracy, because it tells you how much margin you are spending. Static is cruder than dynamic here because its activation ranges were fixed from 200 calibration rows, while dynamic re-measures every input.

Latency: int8 helps most at batch 256 on one thread. Static per-tensor took 155.6 µs and dynamic 211.8 µs, against 680.3 µs for fp32: 4.4x and 3.2x faster. At 4 threads the advantage shrinks (169.9 vs 325.8 µs), because at that point the work is too small to split well. At batch 1 the gain is small (10.6-11.0 vs 15.7 µs), because fixed per-call overhead dominates. Per-channel static was *slower* than per-tensor at batch 1 (16.7 vs 10.6 µs). Granularity has a run-time cost in this kernel path, so do not assume the "more accurate" option is free.

PyTorch dynamic quantization was slow at batch 1 (52.2 µs, against 22.4 µs for fp32 eager). Its per-call overhead (quantizing activations, packed-weight dispatch) outweighs the saved arithmetic at this size. It caught up at batch 256 (267.5 vs 305.1 µs). The lesson: quantization speed-ups depend on the runtime, the batch size and the kernel, so measure on the stack you will ship.

Block 4: Self-check (20 min)

1. Why does the calibration set come from training data? *Test data must stay unseen. Calibrating on it leaks test information into the model's fixed ranges.*
2. Every variant had 0.9741 accuracy. Can you conclude int8 is lossless for your model? *No. Digits has wide margins. Look at max abs ?logit (up to 0.94 here) and test on a task with tight margins.*
3. When would you pick dynamic over static? *When there is no representative calibration data, when input ranges vary a lot, or when you want the low-risk option first. It was also within 0.4 µs of static at batch 1 here.*

4. Why did 4 threads shrink int8's advantage? *The int8 GEMM at batch 256 is already small (about 150-210 µs). Splitting it across threads adds synchronisation cost that eats much of the gain.*

Block 5: Practice (30 min)

Calibrate the static model on only 10 training rows, then on 200 rows that are all one digit class. Compare max abs ?logit and accuracy against today's run. That is how a bad calibration set shows up.

Extra notes

- Symmetric int8 (zero point 0) suits weights, which are roughly centred. Asymmetric uint8 suits ReLU outputs, which are all >= 0, so a symmetric scheme wastes half the range. Our static run used QUInt8 activations and QInt8 weights.
- Percentile calibration clips rare outliers to get a finer scale for the bulk of values. Here it made no measurable difference (0.941 vs 0.935 ?logit).
- For LLMs, weight-only 4-bit quantization (Day 158's 4.5-bit row) is the common choice, because decoding is memory-bound.

Going deeper

Read ORT's quantization page on QDQ versus QOperator and on which execution providers prefer which. Then quantize the Week 21 Day 144 model and compare its accuracy with today's.

Interview-style question (15 min)

"We quantized our model and accuracy dropped 3 points." Possible causes: bad calibration data, per-tensor scales on layers with outlier channels (try per-channel), a sensitive first or last layer (keep it in fp32, i.e. mixed precision), or activation outliers (try percentile calibration or QAT). How would you find which layer is responsible? Quantize one layer at a time and measure.

Common mistakes

- Reporting only accuracy on an easy test set.
- Calibrating on test data.
- Assuming per-channel is always better, or always free.
- Comparing int8 at batch 256 to fp32 at batch 1.

What to do next

Day 157 makes the model smaller another way: fewer weights (pruning) or a smaller student (distillation).

Day 157: Pruning and distillation basics, at toy scale

Review today: Day 156 (+1), Day 154 (+3), Day 150 (+7), Day 136 (+21; quick 15-min skim of its self-check).

Block 1: Two ways to make a model smaller (25 min)

Pruning removes weights. *Unstructured* pruning zeroes individual weights, usually the ones with the smallest magnitude, anywhere in the matrix. *Structured* pruning removes whole units (neurons, channels, attention heads), so the matrices actually get smaller. The distinction matters because ordinary dense kernels multiply zeros at full price. Unstructured sparsity only saves time with sparse kernels or hardware built for a sparsity pattern, and only at high sparsity. It always saves *compressed* file size, because zeros compress well, but that saving disappears once the weights are unpacked into memory.

Distillation trains a small *student* to imitate a large *teacher*. The student learns from the teacher's softened output distribution (softmax at temperature T > 1) as well as from the true labels. The usual loss is ?.T^2.KL(teacher_T || student_T) + (1 - ?).CE(labels). The T^2 keeps gradient scales comparable as T changes. The hoped-for benefit is that the soft targets carry "dark knowledge": which wrong classes are similar to the right one (a 7 looks more like a 1 than like a 0).

Block 2: Run it (35 min)

`python day157_prune_distill.py` takes the Day 155 model (0.9741 accuracy, 796 µs at batch 256 on 1 thread) and runs three experiments. Each pruning variant gets a 5-epoch fine-tune at learning rate 5e-4.

Verified (out/day157_prune_distill.log): - **Unstructured global L1 pruning** (`torch.nn.utils.prune.global_unstructured` across all three Linear layers):

sparsity	acc, no fine-tune	acc after fine-tune	dense b256 µs
0.50	0.9759	0.9741	803
0.80	0.9630	0.9778	785
0.90	0.8704	0.9815	807
0.95	0.2500	0.8889	782
0.98	0.1537	0.1444	783

- **Structured pruning** (keep the top-k hidden neurons by the L2 norm of their incoming weights, in both hidden layers, then fine-tune):

hidden	params	acc before fine-tune	acc after	b256 µs	speed-up
256	85,002	0.9185	0.9778	234	3.4x
128	26,122	0.7426	0.9759	85	9.3x
64	8,970	0.5389	0.9222	44	18.1x
32	3,466	0.2315	0.5870	28	28.8x
16	1,482	0.1889	0.2481	23	35.0x

- **Distillation** (student 64-h-10 with one hidden layer, T = 4, 5 seeds per arm, the same initial weights and batch order in both arms):
- h = 8, 30 epochs, ? = 0.7: labels-only 0.8778 (sd 0.0226) vs distilled 0.8067 (sd 0.0280).
- h = 8, 120 epochs, ? = 0.7: 0.9511 (sd 0.0066) vs 0.9293 (sd 0.0092).
- h = 8, 120 epochs, ? = 0.3: 0.9511 vs 0.9326 (sd 0.0079).
- h = 16, 120 epochs, ? = 0.3: 0.9696 (sd 0.0050) vs 0.9626 (sd 0.0078).
- Distillation was worse on every seed in every setting. The gap shrank from about -7 points to about -0.2 to -1.3 points as training lengthened and ? fell.

Block 3: Interpretation (35 min)

Unstructured pruning: accuracy holds up, speed does not move. At 90% zeros the fine-tuned model scored 0.9815, slightly above the unpruned 0.9741. That is within noise on a 540-row test set, where one row is 0.19 points, but it clearly did not get worse. Latency stayed at 782-807 µs throughout, the same as the dense model, because the dense GEMM multiplies the zeros anyway. Two thresholds stand out. Without fine-tuning, accuracy collapses between 90% and 95% sparsity (0.8704 to 0.2500). With fine-tuning, the model recovers at 95% (0.8889) but not at 98% (0.1444): at 98% the remaining 2% of weights cannot express the function. *Fine-tuning after pruning is not optional.*

Structured pruning is what makes a model faster on a CPU. Keeping 128 of 512 neurons cut parameters from 301,066 to 26,122 (11.5x fewer). After 5 epochs of fine-tuning, accuracy was 0.9759, and batch-256 latency fell to 85 µs (9.3x faster). That is the deployable result of the day, and Day 161 ships it. Below 64 neurons, accuracy falls off a cliff with only 5 epochs of fine-tuning (0.5870 at 32). A smaller model trained longer, or trained from scratch (Day 157's distillation students reached 0.9696 with only 16 hidden units after 120 epochs), would do better. Pruning is a starting point for a small model, not magic.

Speed-up is not proportional to parameter count. At 16 hidden units the model has about 200x fewer parameters, but it is only 35x faster, because fixed costs (Python call, allocation, the 64-input and 10-output layers) take over. The same Amdahl's-law effect showed up in Week 22's profiling.

Distillation did not help here, and that is the honest result. The literature reports gains when the teacher knows something the labels do not: many classes with real similarity structure, noisy or incomplete labels, or extra unlabelled data the teacher can label. Here the student sees the same 1,257 clean training images the teacher saw, and the teacher is only 97.4% accurate, so its soft targets also carry its errors. With a short budget (30 epochs) the KL term, scaled by T^2 = 16 with ? = 0.7, dominates and slows learning of the true labels. That explains why the gap is largest there and shrinks with longer training and smaller ?. To give distillation a fair test, add *unlabelled* inputs (e.g. augmented digits) that only the teacher can label, or use a harder dataset. What not to do is tune ? and T on the test set until distillation "wins".

Block 4: Self-check (20 min)

1. Why did latency not change at 90% unstructured sparsity? *Dense kernels multiply every element, zeros included. The saving needs sparse kernels or a hardware-supported sparsity pattern.*

2. At hidden = 128, parameters fell 11.5x but latency only 9.3x. Why? *Per-call fixed costs and the input/output layers do not shrink. Small kernels also run closer to overhead-bound than compute-bound.*
3. Distillation lost on every seed. Name two changes that might make it help. *Add teacher-labelled unlabelled data; use a harder task with similar classes; lengthen training or lower ?. Pick them before looking at the test set.*
4. Why is "0.9815 at 90% sparsity" not evidence that pruning *improves* the model? *The difference from 0.9741 is 4 test rows out of 540. Without repeats or a confidence interval it is within noise.*

Block 5: Practice (30 min)

Run the structured pruning with 20 fine-tuning epochs instead of 5 and see where the accuracy cliff moves. Then train a 64-128-128-10 model *from scratch* for the same total epochs and compare it with the pruned one. Is pruning better than starting small?

Extra notes

- `prune.remove` makes the mask permanent. Before that call, PyTorch keeps `weight_orig` plus a mask, which makes the model *bigger* in memory.
- Choosing neurons by incoming-weight L2 norm is one simple criterion. Others use activation statistics or gradients. For the second layer we indexed the columns by the neurons kept in layer 1, so the shapes stay consistent.
- Some recent CPU and mobile backends support specific sparse patterns. Check your runtime's docs before relying on it, and measure.

Going deeper

Read the PyTorch pruning tutorial's sections on structured pruning (`ln_structured`) and iterative pruning schedules. Then compare one-shot 90% pruning against five rounds of 37% pruning (0.63^5 ~= 0.1 left) with fine-tuning between rounds.

Interview-style question (15 min)

"We need the model 5x faster on a mid-range phone with at most 1 point of accuracy loss." Plan: profile first (Day 155). Try int8 first (Day 156: about 3-4x on GEMMs here). If that is not enough, prune structurally and fine-tune, trying widths from about 50% downwards. Distil only with unlabelled data the teacher can use. Measure every candidate on the target device against the budget (Day 161).

Common mistakes

- Claiming a speed-up from unstructured sparsity without a sparse kernel.
- Skipping fine-tuning after pruning.
- Reading a 0.7-point difference on 540 test rows as real.
- Tuning distillation hyperparameters on the test set.

What to do next

Day 158 moves from a 300k-parameter MLP to a 135M-parameter LLM and asks the on-device question that matters most: will it fit in memory?

Day 158: Memory and KV-cache budgeting on devices

Review today: Day 157 (+1), Day 155 (+3), Day 151 (+7), Day 137 (+21; quick 15-min skim of its self-check).

Block 1: Where an LLM's memory goes (30 min)

At inference, a decoder-only LLM's memory has four parts. (1) **Weights:** parameters x bytes per parameter. (2) **KV cache:** for every layer, the keys and values of every past token, so the next token does not recompute them. (3) **Activations:** temporary tensors for the current forward pass. They are small during one-token decode, but large during *prefill* (processing the prompt), especially the vocab-sized logits. (4) **Runtime overhead:** the framework, allocator slack, the tokenizer, and the app itself.

The KV cache per token is 2 (K and V) x layers x KV heads x head_dim x bytes. Grouped-query attention (GQA) shares each K/V head among several query heads, so the cache shrinks by the ratio of query heads to KV heads. For a phone with a hard memory limit (Android can kill a background app, or a foreground one that grows too large), these numbers decide whether a model ships at all.

Block 2: Run it (30 min)

`python day158_memory_kv.py` reads SmolLM2-135M-Instruct's `config.json` from the local Hugging Face cache, does the arithmetic, then loads the model in fp32 and measures.

Verified (out/day158_memory_kv.log): - **Config:** hidden size 576, 30 layers, 9 query heads but only 3 KV heads (head_dim 64), MLP intermediate size 1536, vocabulary 49,152, tied embeddings, maximum 8,192 positions. - **Parameter count:** the formula gives 3,540,096 per layer (attention 884,736 and MLP 2,654,208), embeddings 28,311,552 (21% of the total), and a total of 134,515,008. `model.parameters()` gives 134,515,008, a difference of +0. - **Weight memory:** fp32 513 MiB; bf16/fp16 257 MiB; int8 128 MiB; 4-bit with one fp16 scale per 32 weights (4.5 bits/weight, an assumed scheme) 72 MiB. - **KV per token:** 23,040 B (22.5 KiB) in fp16 with GQA; a hypothetical 9-head MHA would need 69,120 B (3x); int8 KV would need 11,520 B. In fp16 at batch 1, a context of 512 / 2048 / 8192 tokens needs 11.2 / 45.0 / 180.0 MiB, against 33.8 / 135.0 / 540.0 MiB with MHA. - **Measured:** loading fp32 took 1.4 s, and RSS rose from 390 to 1,020 MiB (+630 MiB, against 513 MiB of fp32 weights). Prefill at 256 / 1024 / 2048 tokens produced real `past_key_values` of 11.2 / 45.0 / 90.0 MiB in fp32. These match the formula exactly, with per-layer K shape (1, 3, ctx, 64). RSS rose by +76 / +302 / +468 MiB, of which the fp32 logits alone were 48 / 192 / 384 MiB. - **Budget table** (ASSUMED app budgets; weights plus fp16 KV only): at 256 MiB, bf16 weights (257 MiB) do not fit at all, int8 leaves room for 5,812 tokens, and 4.5-bit fits the full 8,192. At 512 MiB or more, every format fits the full 8,192.

Block 3: Interpretation (35 min)

The formula is exact, and that is the point. Parameter count, weight bytes and KV bytes are all predictable from `config.json`, before you download a single weight. The measured KV tensors matched the formula to the 0.1 MiB, so budget on paper first and verify once.

GQA is the biggest single KV saving in this model. Three KV heads instead of nine cut the cache 3x: 180 MiB instead of 540 MiB at 8,192 tokens. That is an architecture choice made at training time, which you inherit when you pick a model. When comparing candidate models for a phone, compare their `num_key_value_heads x head_dim x layers`, not just their parameter counts.

Prefill logits are the hidden memory spike. At 2,048 tokens of prefill, RSS rose by 468 MiB, of which 384 MiB was the fp32 logits tensor (2048 x 49,152 x 4 bytes) for *every* prompt position. Generation only needs the last position's logits. Production runtimes compute just that one, and they process long prompts in chunks. Day 158's naive `m(ids)` call shows what happens if you don't. The other ~84 MiB is the KV cache (90 MiB) plus other temporaries, minus allocator effects. RSS deltas are approximate because the allocator reuses freed memory.

RSS after loading exceeded the weights by about 117 MiB (630 vs 513 MiB): library code, allocator slack, and the loading path's temporary buffers. On a phone the runtime is smaller than PyTorch, but there is still overhead. Budget a margin, and measure the peak on the device (Android Studio's memory profiler) rather than trusting the arithmetic alone.

How to read the budget table. The budgets (256/512/1024 MiB) are *assumptions* for illustration, not platform limits: real limits depend on the device's RAM and what else is running, so measure on your lowest-end target device. The table's lesson is about ratios. At a tight 256 MiB budget, the weight format decides whether the model ships at all (bf16 cannot; int8 can, with a 5,812-token context). Once weights are 4-bit, the KV cache becomes the main cost, and int8 KV (11,520 B/token) or a capped context is the next lever. llama.cpp's Android doc warns about exactly this: start with a modest context size, or memory can spike.

Block 4: Self-check (20 min)

1. Compute the fp16 KV per token for a model with 32 layers, 8 KV heads and head_dim 128. *2 x 32 x 8 x 128 x 2 = 131,072 B = 128 KiB per token; 4,096 tokens = 512 MiB.*
2. Why were the measured KV tensors 90.0 MiB at 2,048 tokens when the formula said 45.0 for fp16? *We loaded in fp32 (4 bytes). The fp32 formula row gives 90.0, an exact match.*
3. What is the cheapest fix for the 384 MiB logits spike? *Compute logits only for the last position (or chunk the prefill). It costs no accuracy at all.*

4. With 4.5-bit weights at a 256 MiB budget, which part dominates memory at 8,192 tokens? *The KV cache: 180 MiB fp16 against 72 MiB of weights.*

Block 5: Practice (30 min)

Extend the script with an int8-KV column for the budget table. Then pick any other small model config you already have locally and fill in the same table without loading its weights.

Extra notes

- Batch size multiplies the KV cache. On-device apps are almost always batch 1, while servers pay for batch x context.
- Tied embeddings (input and output share one matrix) save 28.3 M parameters here, 21% of the model.
- Sliding-window attention, KV quantization and paged KV caches are further levers. Each trades complexity or quality for memory.

Going deeper

Measure decode (one token at a time with `past_key_values`) RSS growth per token and check it against 23,040 x 2 B per token (fp32). Then try `torch.float16` or `bfloat16` loading and see how the RSS after load changes.

Interview-style question (15 min)

"Will a 1B-parameter model with 16 layers, 8 KV heads and head_dim 64 fit a 1 GiB budget with a 4k context?" Weights at 4.5 bits ~= 536 MiB. KV per token in fp16 = 2 x 16 x 8 x 64 x 2 = 32 KiB, so 4,096 tokens = 128 MiB. That is about 664 MiB plus activations and runtime overhead, so it probably fits. But measure the peak during prefill, and keep a margin for the app itself.

Common mistakes

- Counting weights but not the KV cache.
- Using MHA arithmetic for a GQA model (3x too pessimistic here).
- Forgetting prefill logits.
- Reading the RSS after load as "the model's size".

What to do next

Day 159 takes the model files from this week and puts them through the three Android runtimes' toolchains.

Day 159: Shipping to Android: ONNX Runtime Mobile, ExecuTorch, LiteRT and llama.cpp

Review today: Day 158 (+1), Day 156 (+3), Day 152 (+7), Day 138 (+21; quick 15-min skim of its self-check).

Block 1: The shape of every on-device pipeline (25 min)

You are an Android developer, so you know the app half: Gradle dependencies, assets or downloaded files, a background thread, JNI when native code is involved. The ML half has the same shape whichever runtime you pick. **(1) Export** offline from the training framework to the runtime's file format. **(2) Optimise and quantize** offline (Days 155-157). **(3) Package** the model in the APK/AAB as an asset, or download it after install (big models usually are). **(4) Load** it once, with a runtime library from Maven, into a session/module object. **(5) Run** it per input from a worker thread, with the input converted from your app's types into the runtime's tensor type. **(6) Measure** latency, memory and battery on real target devices.

The four options covered today, each described only from its official page (read 2026-10-29):

- **ONNX Runtime Mobile.** The ORT install page's Android section gives the Gradle dependency `com.microsoft.onnxruntime:onnxruntime-android` from Maven Central. For C/C++ use, it says to unzip the same AAR and take the headers and `libonnxruntime.so` from it. Models can be plain `.onnx` or the ORT format (`.ort`), which ORT's docs recommend for mobile and minimal builds. The ORT docs also describe a *custom build* that includes only the operators your models use, to shrink the library. The Java API (from the ORT Java getting-started page): `OrtEnvironment.getEnvironment()`, `env.createSession(path, OrtSession.SessionOptions())`, `OnnxTensor.createTensor(env, buffer, dims)`, `session.run(mapOf(name to tensor))`.

- **ExecuTorch** (PyTorch's on-device runtime). The "Using ExecuTorch on Android" page gives the Maven dependency `org.pytorch:executorch-android:${executorch_version}`. Its AAR contains the Java library plus the JNI `.so`, in arm64-v8a and x86_64 variants, and bundles the XNNPACK CPU backend. The page lists XNNPACK (CPU), Vulkan (GPU), Qualcomm and MediaTek (NPU) backends and says to start with XNNPACK. Its Java sample uses `Module.load(path)`, `Tensor.fromBlob(...)`, `EValue.from(...)`, `module.forward(...)[0].toTensor().getDataAsFloatArray()`. The model file is a `.pte`.
- **LiteRT** (the new name of TensorFlow Lite). Google's LiteRT Android page shows `com.google.ai.edge.litert:litert:2.3.0` (and optional `litert-gpu`), and a Kotlin `CompiledModel.create(path, CompiledModel.Options(Accelerator.CPU))`, `createInputBuffers()`, `writeFloat`, `run`, `readFloat` flow. The model file is a `.tflite`.
- **llama.cpp.** Its `docs/android.md` describes an Android Studio project (`examples/llama.android`) whose Kotlin binding wraps the native library. It loads GGUF models from app-private storage. The doc also covers Termux builds and NDK cross-compilation with CMake (`-DANDROID_ABI=arm64-v8a`, `-DGGML_NATIVE=OFF`, and so on). It is the natural choice for LLMs in GGUF format; the other three suit general models.

Block 2: Run what can be run here (35 min)

`python day159_android_export.py` produces each runtime's model file and runs it through the runtime's Python API **on this x86 Linux box**.

Verified (out/day159_android_export.log): - **ORT format:** `python -m onnxruntime.tools.convert_onnx_models_to_ort` exited 0. It wrote `w23_mlp.ort` (1,208 kB), `w23_dyn.ort` (311 kB), `.with_runtime_opt.ort` variants (1,208 and 315 kB) and two `required_operators*.config` files. - Loading the `.ort` files back: fp32 accuracy 0.9741 with max diff vs torch 0.000e+00; int8 accuracy 0.9741 with max diff 1.323e-01, the same as Day 156's dynamic model. - Latency at batch 1 (1 thread): 20.1 µs for `w23_mlp.ort`, 17.2 µs for its runtime-opt variant, 10.8 µs for `w23_dyn.ort` and 15.1 µs for its runtime-opt variant. - The operator config lists exactly `ai.onnx;13;Gemm`, `ai.onnx;14;Relu` and `com.microsoft;1;DynamicQuantizeMatMul,FusedGemm`. That is the input for a reduced-operator custom build. - **ExecuTorch 1.5.1:** `torch.export` -> `to_edge_transform_and_lower` -> `to_executorch` produced two `.pte` files of 1,207 kB each, one without a delegate (portable kernels) and one with the XNNPACK partitioner. - Both ran in the ExecuTorch Python runtime with 0.9741 accuracy, and max differences vs torch of 2.098e-05 (portable) and 2.480e-05 (XNNPACK). - Batch-1 latency: portable 1,120.4 µs; XNNPACK 10.7 µs. Eager torch at 1 thread took 29.4 µs. - **LiteRT:** the `ai_edge_litert` 2.3.0 `Interpreter` imports. Conversion from PyTorch is **UNTESTED** (converter not installed).

Block 3: Interpretation, and the UNTESTED app side (35 min)

The files are real; the phone numbers are not. Today proves that our exported model converts cleanly to `.ort` and `.pte`, that the converted files load in the same runtime libraries that ship in the Android AARs, and that they give the right answers. It does **not** say how fast they are on a phone. The µs figures are from a server Xeon. Run them again on your devices.

The backend matters more than the format. The same ExecuTorch graph took 1,120.4 µs on portable kernels and 10.7 µs with XNNPACK, about 100x. Portable kernels are reference C++ meant for correctness and portability. XNNPACK is the optimised CPU library. In an app, a forgotten delegate looks exactly like "ExecuTorch is slow". The same goes for ORT's `.with_runtime_opt` variants: they did not help this tiny model (17.2 vs 20.1 µs fp32, but 15.1 vs 10.8 µs int8), so test both.

Size and operator coverage. The `required_operators.config` lists exactly four kernels (Gemm, Relu, DynamicQuantizeMatMul, FusedGemm). A custom ORT build with only those is the documented way to cut library size, which matters when the runtime library is many times bigger than a 307 kB model. Its size on a device is **UNTESTED** here.

The UNTESTED Kotlin side lives in `code/week-23/android_untested/`: - `build.gradle.kts.snippet`: the three Maven coordinates from the pages above, with versions left as `<version>`. - `OrtDigits.kt`, `EtDigits.kt` and `LiteRtDigits.kt`: one small class per runtime, each wrapping a `predict(FloatArray): Int` around the documented calls. Input shape is [1, 64], with pixels/16 as in `_dep.py`. - `llama_cpp_NOTES.md`: notes on llama.cpp.

None of it has been compiled. Each class should run off the UI thread: the inference call blocks, and a 1,000-row loop on the main thread would freeze the app. One gap is flagged in the ORT file: the method for setting intra-op threads in Java is not shown on the pages I checked, so it is left as a TODO rather than guessed.

How to choose. - Already in ONNX, or the model comes from scikit-learn, TensorFlow or other tools? ORT Mobile. - A PyTorch-native team that wants the NPU/GPU backends listed above? ExecuTorch. - TensorFlow/Keras models, or Google's accelerator path? LiteRT. - An LLM in GGUF format? llama.cpp. - Every runtime: test on the lowest-end target device first.

Block 4: Self-check (20 min)

1. Why does a `.pte` without a delegate run 100x slower here? *Portable kernels are simple reference implementations. The XNNPACK delegate replaces the graph's ops with optimised CPU kernels (SIMD, blocking).*
2. What does `required_operators.config` let you do? *Build a custom ORT runtime with only the listed operators, which shrinks the native library you ship in the APK.*
3. Name two things a phone run would change that this box cannot tell you. *Absolute latency (clocks, cache, thermal throttling), memory limits and battery cost, and whether the NPU/GPU delegates work for your ops.*
4. Where should the model file live for a 300 MB LLM? *Not in the APK. Download it after install into app-private storage, and check its hash.*

Block 5: Practice (30 min)

If you have Android Studio: create an empty app, add the ORT dependency, copy `w23_dyn.ort` into assets, adapt `OrtDigits.kt`, and log `predict` on one test row. The expected class is printed by the Day 159 Python code for row 0 (`Xte[0]`). Then record the latency of 1,000 calls on your phone next to Day 161's table. Mark every phone number with the device model.

Extra notes

- The ExecuTorch `.pte` here was exported with a static batch of 1, which is typical for on-device use. Dynamic shapes are possible, but need extra export settings.
- The `x86_64` ABI in ExecuTorch's AAR is what an emulator on a PC uses, so emulator timings are not phone timings either.
- NNAPI was the old Android-wide accelerator API (the original SYLLABUS row mentioned it). Today's pages point to per-vendor NPU backends and delegates instead. Check each runtime's current docs before relying on any accelerator.

Going deeper

Read ORT's "ORT format models" page and the ExecuTorch getting-started page, and write down what each says about operator coverage and model compatibility across versions. Those are the things that break app updates.

Interview-style question (15 min)

"Our on-device model is fast in the benchmark app but janky in the real app." Possible causes: inference running on the UI thread; a model reloaded per call instead of once; allocations per call (reuse input buffers); CPU contention with rendering (too many intra-op threads); thermal throttling after minutes of use; or the delegate failing to initialise and silently falling back to CPU. How would you check each with the profiler and logs?

Common mistakes

- Exporting without the XNNPACK partitioner and blaming the runtime.
- Shipping a hardware-specific optimised graph (Day 155's warning).
- Copying a Maven version from a blog instead of the official page.
- Calling inference on the main thread.

What to do next

Day 160 opens up the engine itself: a small MLP inference engine written in Rust, checked against PyTorch and ORT.

Day 160: A tiny inference engine in Rust (PyO3), checked against PyTorch and ONNX Runtime

Review today: Day 159 (+1), Day 157 (+3), Day 153 (+7), Day 139 (+21; quick 15-min skim of its self-check).

Block 1: Why write one (20 min)

You will rarely ship a hand-written engine. ORT, ExecuTorch and LiteRT exist for good reasons. Writing one, though, teaches what they do for you: weight layout, quantization arithmetic, accumulation types, threading, and the difference between matrix-vector and matrix-matrix work. Rust is a good fit for on-device native code: no garbage collector, memory safety, and it

compiles for Android ABIs. It also connects to Week 22's PyO3 kernels (Day 153) and Week 17's int8 matmul crate.

Block 2: The engine and the run (40 min)

`code/week-23/tiny_infer/src/lib.rs` defines two Python classes: - **`MlpF32(ws, bs):`** copies the row-major [out, in] weights. `forward(x, parallel)` runs each input row through every layer as a dot product per output neuron (16 f32 accumulators, so the compiler can vectorise), then adds the bias and applies ReLU. - **`MlpInt8(ws, bs):`** quantizes the weights at construction time, per output row and symmetric: s = max|w_row|/127, q = round(w/s). At run time it quantizes each activation row symmetrically (scale = max|x|/127), accumulates in i32, then dequantizes with x_scale x w_scale[row] and adds the fp32 bias. - With `parallel=True`, rows are split across threads with rayon.

It is built with `RUSTFLAGS="-C target-cpu=native" CARGO_NET_OFFLINE=true maturin develop --release`. The `day160_rust_engine.py` script checks the engine against PyTorch, against a NumPy re-implementation of the same int8 algorithm, and against ORT.

Verified (out/day160_rust_engine.log): - Rust fp32: max |diff| vs torch 1.14e-05, accuracy 0.9741, weights 1,204 kB. - Rust int8: max |diff| vs the NumPy same-algorithm version 0.00e+00. Serial and parallel outputs are identical (`True`). Max |?logit| vs fp32 is 0.162, accuracy 0.9741, argmax agreement with fp32 1.0000, weights 308 kB. - Rust int8 vs ORT dynamic int8 logits: max |diff| 0.180. They use different activation quantization schemes, so they are not bit-identical. - Median latency in µs/call:

engine	batch 1	batch 256
torch eager fp32 (1 thread)	30.1	782.9
ORT fp32 (1 thread)	14.7	683.8
ORT dynamic int8 (1 thread)	10.8	212.8
Rust fp32 serial	17.0	4,279.5
Rust int8 serial	31.3	8,058.8
Rust fp32 rayon (8 threads)	16.9	642.7
Rust int8 rayon (8 threads)	31.3	1,072.4

Block 3: What the numbers teach (35 min)

Numerics first, and they pass. Rust fp32 differs from torch by 1.14e-05. That is a different summation order (16 partial sums instead of a BLAS kernel's order), the normal fp32 reassociation error, and it would fail an exact-equality test for no good reason. Rust int8 matches the NumPy version of the same algorithm *exactly*. That is the strongest test for a quantized kernel: write the algorithm twice, independently, and require identical integers. Its error vs fp32 (0.162) is the quantization error itself, comparable to ORT dynamic's 0.132 on Day 156.

At batch 1, a simple engine is competitive. Rust fp32 at 17.0 µs beat eager torch (30.1) and came close to ORT fp32 (14.7). With one input row, the work is three matrix-vector products. Its speed is set by how fast the 1.2 MB of weights stream from cache, and by per-call overhead, both of which are small in a few hundred lines of Rust.

At batch 256, the simple engine loses badly, and the reason is the most important idea of the day. Rust fp32 serial took 4,279.5 µs, 6x slower than ORT fp32 at 683.8. The engine processes one row at a time, so it re-reads every weight from memory 256 times. A real GEMM *blocks* the computation: it loads a tile of weights into registers and cache, and reuses it across many rows. That turns a memory-bound loop into a compute-bound one, which is Week 22's roofline (Day 152) in practice. Rayon across 8 threads hid much of it (642.7 µs, slightly faster than 1-thread ORT), but only by spending 8 cores.

Our int8 was slower than our fp32 (31.3 vs 17.0 µs at batch 1). ORT's int8 was faster than its fp32. The difference is the kernel. Our i8xi8->i32 loop widens each element to i32 before multiplying. That vectorises poorly, and it does not reach the CPU's VNNI dot-product instructions (present on this box), which ORT's MLAS kernels are built to use. Quantization saves memory automatically, but it saves *time* only when the kernel uses the hardware's integer instructions. That is why you use a runtime and don't write your own.

Block 4: Self-check (20 min)

1. Why is "max |diff| 0.00 vs a NumPy re-implementation" a stronger test than "accuracy unchanged"? *It checks every integer and scale in every layer. Accuracy can hide large errors on an easy task.*

2. Why does a row-at-a-time engine lose at batch 256 but not at batch 1? *At batch 1 there is nothing to reuse. At batch 256 a blocked GEMM reuses each weight tile across many rows, while the row loop streams all the weights 256 times.*
3. Why could int8 be slower than fp32? *The inner loop's widening conversions, and no use of VNNI/dot-product instructions. Speed comes from the kernel, not the data type.*
4. Why was serial vs parallel output identical? *Each row is computed independently in the same order. Threads only change which core does a row, not the arithmetic within it.*

Block 5: Practice (35 min)

Add a `forward_batched` that processes 8 rows per weight-row pass, accumulating 8 dot products at once while each weight row is in cache. Measure batch 256 again. You should see the serial time drop. Then try an `i16` accumulation of pairs before widening to `i32` in `dot_q`, and check that the NumPy comparison still gives 0.00.

Extra notes

- On Android, a Rust crate is built per ABI (`aarch64-linux-android` etc.) and exposed to Kotlin through JNI. This needs the NDK and is **UNTESTED** here. The binding pattern (thin wrapper, no allocations in the hot path, release locks/threads during compute) is the same as PyO3's `allow_threads`.
- `-C target-cpu=native` tunes for *this* CPU. For distribution you would target a baseline (e.g. armv8-a with dotprod) and pick kernels at run time, which llama.cpp's Android doc describes doing.
- Week 17's `rust_qmatmul` crate and this one use the same quantization idea; this one adds per-row scales and dynamic activation scales.

Going deeper

Read how a blocked GEMM is structured (any BLAS kernel explanation, or Week 22's roofline notes), then sketch the tile sizes that would fit this CPU's L2 for a 512x512 int8 weight matrix.

Interview-style question (15 min)

"Why not just write our own inference code? It's only matrix multiplies." Answer with today's data: correct numerics are achievable (we matched to 0.00 against an independent version), but performance needs blocked, SIMD- and VNNI-aware kernels for every shape, plus threading, operator coverage, quantization schemes and per-device tuning. That took our engine from 6x slower to merely competitive only at batch 1. Custom code is justified for a small, fixed model with special constraints, or one unsupported operator. Otherwise, use a runtime.

Common mistakes

- Comparing floats for exact equality across engines.
- Concluding "int8 is slower" from a naive kernel.
- Benchmarking a parallel engine against single-threaded baselines without saying so.
- Forgetting `--release` (debug builds are many times slower).

What to do next

Day 161 puts everything into one table under one budget, and picks what to ship.

Day 161: PROJECT W23: an on-device-style inference pipeline under a latency/memory/accuracy/size budget; Quiz 23

Review today: Day 160 (+1), Day 158 (+3), Day 154 (+7), Day 140 (+21; quick 15-min skim of its self-check).

Block 1: The brief (20 min)

Ship a digit classifier that a phone app calls once per input: batch 1, one intra-op thread (so it does not fight the UI for cores). The budget is fixed *before* measuring, as it would be in a product spec: - test accuracy >= 0.97; - p95 latency <= 20 µs on this box; - model file <= 400 kB; - memory growth from loading <= 30 MiB.

The p95 figure, not the median, is what users feel as jank. The budget values are this project's choices for this box, not phone benchmarks. On a device you would rescale them after measuring the baseline there.

Block 2: Build and measure (45 min)

`python day161_project.py` rebuilds the artefacts it needs: the structured-pruned 128-unit model from Day 157 (5-epoch fine-tune), its ONNX export, and dynamic int8 versions of both models. It then measures nine variants. Each variant runs **in a fresh subprocess**, so the RSS growth of its load (runtime import plus model) is not polluted by the others. In each one it scores all 540 test rows one at a time, warms up with 50 calls, and times 1,000 single-row calls.

Verified (out/day161_project.log):

variant	file kB	acc	p50 µs	p95 µs	load RSS MiB	verdict
torch eager fp32	1207	0.9741	38.6	51.7	3.3	FAIL: p95, size
ORT fp32	1205	0.9741	15.1	15.4	9.9	FAIL: size
ORT dynamic int8	307	0.9741	10.9	11.4	7.7	PASS
ORT static QDQ int8	308	0.9741	10.6	12.9	8.1	PASS
ORT-format .ort int8	312	0.9741	11.1	12.7	7.8	PASS
ExecuTorch XNNPACK fp32	1208	0.9741	12.1	14.9	134.9	FAIL: size, rss
Rust int8 (tiny_infer)	308	0.9741	31.2	35.8	3.1	FAIL: p95
pruned-128 ORT fp32	105	0.9759	5.2	6.5	7.2	PASS
pruned-128 ORT dyn int8	30	0.9759	7.5	9.4	7.4	PASS

The Rust row's "file" size is the in-memory int8 weights plus scales and biases (308 kB, from `MlpInt8.nbytes()`). It loads the fp32 `.pt` and quantizes at start-up.

Block 3: Decide, and say what the table cannot tell you (40 min)

Pick: pruned-128 + ORT, in ORT format. It passes everything by a wide margin. The fp32 version is 105 kB with p95 6.5 µs, the fastest variant measured. Its int8 version is only 30 kB with p95 9.4 µs. Both scored 0.9759, two rows better than the full model, which is within noise. If size matters most (an over-the-air model update, say), ship pruned int8. If latency matters most, ship pruned fp32. Notice that int8 made the *pruned* model slower (7.5 vs 5.2 µs p50). At 26k parameters, the per-call cost of quantizing the activations outweighs the cheaper GEMMs. Day 156's speed-up was for a 300k-parameter model at batch 256. Quantization is not a universal speed-up; it depends on the model size and the batch.

The three full-size int8 ORT variants also pass (p95 11.4-12.9 µs, about 307-312 kB). They are the fallback when you cannot retrain, since pruning needs a fine-tune and quantization does not.

Why the others fail, and what each failure teaches. - *Torch eager* fails on p95 (51.7 µs): framework overhead per call. It also fails on size. - *ORT fp32* is fast (p95 15.4 µs), but its fp32 file is 1,205 kB. - *ExecuTorch XNNPACK* is fast too (p95 14.9 µs), but it is fp32 (we did not quantize the `.pte`), and its "load" added 134.9 MiB. That figure measures *importing the ExecuTorch Python package* in a process that already had torch, not what the ~MB-scale Android runtime would cost. It is an honest measurement of the wrong thing for a phone. The table must say so, rather than quietly drop the row. Quantizing for ExecuTorch (its XNNPACK quantizer flow) is the next experiment, and it is **UNTESTED** here. - *Rust int8* fails on p95 (35.8 µs): Day 160's naive kernel. It does have the smallest load footprint (3.1 MiB), which is the thing a hand-written engine really does well.

Limits of this table. - It runs on a server CPU, not a phone. Re-measure on real devices, and include a low-end one. - RSS deltas include importing each runtime's Python package, not the size of the Android native library. - p95 over 1,000 calls in a quiet process does not capture thermal throttling or contention with an app's UI. - Accuracy is on 540 easy test rows, where one row is 0.19 points. - There is no battery measurement.

Each of these is a line in the device-test plan, not a reason to skip the table.

Block 4: Self-check (15 min)

1. Why measure each variant in a fresh process? *So that one variant's imports and allocations don't count towards another's memory, and caches start in the same state for all.*
2. Why does the budget use p95 instead of the median? *Users notice the slow calls. The p50 can look good while p95 misses the frame budget.*
3. Why was int8 slower than fp32 for the pruned model? *At 26k parameters, the fixed cost of quantizing activations per call is larger than the arithmetic saved.*
4. The ExecuTorch row's 134.9 MiB: bug, or real? *A real measurement of importing the Python package, not of the Android runtime. Report it with that label.*

Block 5: Write-up and extension (40 min)

Write a one-page "model card for deployment": the chosen variant; the budget table; the test conditions (CPU, threads, batch, number of calls); the known limits listed above; and the device-test plan. Then, if you have a phone: run `OrtDigits.kt` (Day 159) with `w23_pruned128.onnx` converted to `.ort`, and add a column of real device p50/p95 values with the phone model named.

Extra notes

- Choose the budget before measuring. Choosing it afterwards makes any result look like a pass.
- Day 161's p50 for ORT dynamic int8 (10.9 µs) matches Day 156's median (11.0 µs) and Day 159's `.ort` value (10.8 µs). Consistency across scripts and days is a cheap check that the bench is stable.
- Keep the artefacts reproducible. Day 161 rebuilds the pruned model with fixed seeds, and `run_all.sh 6` re-runs the whole week.

Going deeper

Add two columns: start-up time (session creation in ms) and first-call latency. On a phone, cold start often matters more than steady-state p95.

Interview-style question (15 min)

"Present this table to a product manager in two minutes." Example: "We can ship a 30-105 kB model with 97.6% accuracy at under 10 µs per input on our test CPU, with about 7 MiB of memory to load. It is the pruned model running on ONNX Runtime. Next we need numbers from three real phones, including a low-end one, plus a battery check, before we commit."

Common mistakes

- Dropping failing rows instead of explaining them.
- Mixing batch sizes or thread counts between rows.
- Calling server-CPU numbers phone numbers.
- Choosing a budget after seeing the results.

What to do next

Take the weekly quiz below. Week 24 continues Month 6: quantization-aware training and llama.cpp on edge devices (SYLLABUS Days 162-163). Day 158's KV arithmetic and Day 160's kernel lessons are the starting points.

Weekly quiz: Week 23 (12 questions; 40 minutes; covers Days 155-161)

1. Day 155's saved graphs at EXTENDED and ALL contained `FusedGemm`. What was fused, and why did it barely change latency?
2. On Day 155, what happened to batch-256 latency going from 1 to 4 to 8 intra-op threads, and what is the deployment lesson for batch 1?
3. Why does ORT warn about saving a model optimised at ENABLE_ALL?
4. On Day 156, which int8 variant was fastest at batch 256 on 1 thread, and how much faster than fp32?
5. Every Day 156 variant scored 0.9741. What other number shows that quantization did change the model, and why does it matter?

6. Explain the difference between dynamic and static quantization in terms of when activation scales are decided.
7. On Day 157, why did 90% unstructured sparsity leave latency unchanged, while keeping 128 of 512 neurons made batch 256 about 9x faster?
8. Distillation lost on every seed on Day 157. Give the most likely reason, and one fair way to test distillation again.
9. Compute SmolLM2-135M's fp16 KV cache for 4,096 tokens from its config, and say how much a 9-head MHA would need.
10. On Day 158, what caused most of the +468 MiB RSS growth at a 2,048-token prefill, and how do runtimes avoid it?
11. Why was the ExecuTorch portable `.pte` about 100x slower than the XNNPACK one?
12. Why was the Day 160 Rust engine competitive at batch 1 but 6x slower than ORT at batch 256, and why was its int8 path slower than its fp32 path?

Answers

1. Each Gemm+Relu pair became one `FusedGemm` (Gemm with the activation built in). The Relu pass over the activations is tiny next to the GEMM (97% of kernel time in the profiler), so removing it saves almost nothing.
2. 676.3 -> 325.1 -> 313.9 µs: about 2.1x by 4 threads, then flat. At batch 1, 4 threads gave 11.3 against 15.6 µs. That gain is not worth using four cores on a phone, so default to 1-2 threads and measure.
3. The ALL level can apply hardware-specific layout transforms (NCHWc) that suit this machine. Ship an unoptimised or EXTENDED graph, and let the device's runtime optimise for itself.
4. Static QDQ per-tensor MinMax: 155.6 µs against 680.3 µs for fp32, about 4.4x. Dynamic int8 took 211.8 µs (3.2x).
5. Max |?logit|: 0.132 for dynamic and about 0.94 for static. Digits has wide decision margins. On a task where the top classes are close, perturbations that size would flip predictions.
6. Dynamic measures each activation's range at run time, on every call: no calibration, small run-time cost. Static fixes activation ranges ahead of time from a calibration set: faster, but only as good as that data.
7. Dense kernels multiply zeros too, so unstructured sparsity saves nothing without sparse kernels (782-807 µs at every sparsity). Structured pruning shrinks the matrices (26,122 parameters against 301,066), so batch 256 took 85 µs against 796 µs.
8. The teacher (0.9741) has no extra knowledge: the student trains on the same clean labelled data, and with a short budget the T^2-scaled KL term slows learning of the labels. A fair re-test adds teacher-labelled *unlabelled* data or uses a harder dataset, with ? and T chosen before looking at the test set.
9. 2 x 30 layers x 3 KV heads x 64 x 2 bytes = 23,040 B per token; x 4,096 = 90 MiB (94,371,840 B). MHA with 9 KV heads would need 3x that: 270 MiB.
10. The fp32 logits for every prompt position: 2,048 x 49,152 x 4 B = 384 MiB. Runtimes compute logits only for the last position, and process the prompt in chunks.
11. Portable kernels are simple reference implementations. The XNNPACK delegate runs the ops with optimised SIMD CPU kernels: 1,120.4 vs 10.7 µs.
12. At batch 1 there is nothing to reuse, so a simple dot-product loop is close to optimal (17.0 vs ORT's 14.7 µs). At batch 256 it re-reads every weight for every row, while a blocked GEMM reuses weight tiles from cache (4,279.5 vs 683.8 µs). Its int8 loop widens each value to i32 and does not use the VNNI dot-product instructions, so int8 saved memory (308 kB) but not time (31.3 vs 17.0 µs).